

Abstract- This report reports on a security test of a Java Swing Library Management System which communicates using JDBC to a MySQL backend. The five user roles are Admin, Librarian, Customer, Researcher and Guest and the application is used to cover book inventory, loan requests, peer to peer transactions, restricted collection access as well as an administrative audit view. My practice involved a manual code review motivated by the OWASP Testing Guide v4.2 and an OWASP ASVS 4.0 checklist, and targeted dynamic testing of a running database. After reviewing the original code, ten already known vulnerabilities were found as well as five additional exploitable defects were deliberately added to another 02_Vulnerable build, including SQL injection and two business logic vulnerabilities, a broken access control bug and a verbose information disclosure. All the vulnerabilities were addressed in 03_Secured with a defense in depth approach based on five collaborating layers and all fixes were aligned to OWASP Top 10:2025, CWE, ASVS 4.0, NIST SSDF and where applicable, CERT Oracle Coding Standard of Java. Validation and regression testing It is verified that all exploits in the vulnerable version are prevented by the secured version and legitimate operations are maintained

Keywords - Java security, OWASP Top 10:2025, ASVS 4.0, CWE, SQL injection, access control, BCrypt, defense in depth, NIST SSDF, JDBC, business logic.

1. Introduction

The test system is a Java Swing desktop application to operate a small university library and all persistence is via MySQL via JDBC. By logging in, users choose a role, and the latter opens its own panel with its features. Admin handles the users and books, Librarian processes the loans and exchanges, Customer processes the borrowing and swapping requests, Researcher processes the restricted collection and Guest has read only access to the catalogue. The architecture is two tier in that Swing communicates directly with the DAO, and the DAO communicates directly with MySQL, so a trust boundary is effectively placed between the GUI and

the DAO and all security choices must be implemented by the DAO as the user has control of the JVM. In the review, I took into account two attacker profiles. The former is an authorized user misusing the business process, such as making multiple loan applications on the same book, making a title application that was never borrowed, reading an embargoed research article prior to its publication. The second is a remote attacker who is only armed with a handcrafted login string since this is a Swing application the normal web attack surface is lost and SQL injection into the login and search form is still highly feasible and is precisely the vulnerable point that a static reviewing tool will point out. The report takes a tour of the state of the art, the testing methodology, the ten existing findings previously, the five injected vulnerabilities, the remediation, and the validation.

2. State of the art in secure programming paradigms

The three paradigms continually reappear in the secure development literature which are secure by design, defensive programming and formal verification, and trades rigour to practicality in various forms [9] . Secure by design is about making security choices in the architecture such that misuse is difficult by default, and McGraw posits that architectural risk analysis will detect more dangerous defects than pure code level analysis since many bugs only have sense at the component boundary. Defensive programming assumes that all callers and all inputs are hostile and authenticates pre-conditions early enough to ensure that bad state does not spread, and Bloch provides a set of clean Java specific guidelines to the same in Effective Java. The most demanding, but not generally implementable with application level Java, is formal verification; which is very costly so I employed an OWASP compliant manual review as the main tool and regarded dynamic testing as an assurance measure as opposed to a substitution. The three popular buckets of security testing paradigms each have a distinct role in a Java code base. Static Application Security Testing analyses the source or the bytecode without running it and also its is good at pattern bugs such as string concatenation into SQL with tools adding over around four hundred rules. Dynamic Application Security Testing executes the developed artifact and tests it

externally, that is, in a Swing case, by submitting malicious data via the actual GUI or by simulating the JDBC calls. Interactive Application Security Testing is based on runtime instrumentation and is highly accurate but not particularly well applicable to small desktop apps that have no HTTP surface. The same blind spot is experienced by all the three paradigms that is business logic since no pattern matcher will ever recognise that requestLoan must also test stock or that a restricted book should not be presented to a researcher until after the embargo date has elapsed and therefore a systematic manual audit against ASVS 4.0 [1] is necessary.

Java and OOP are a half-baked lot as far as security is concerned and it is worth writing that down since it will influence every design choice of this project. Encapsulation allows the DAO to keep the validation back-off behind a clean interface to ensure that callers do not bypass the check, strong typing prevents the buffer overflow and pointer arithmetic failure modes of C and C++ and the JVM provides a managed memory model and array bounds checking which removes an entire category of memory corruption bugs at run time [4], [9]. String immutability, on the other hand, implies that an accessible password is stored as a String on the heap to be accessed by a garbage collector until it is cleared (CWE-316), reflection and serialization may be employed to violate access modifiers and allow a private field to be accessed outside the class (CWE-915), and shared mutable state between threads may silently break an invariant (CWE-665). The reason behind the secured version using char as passwords and setting them to zeros after each use according to the CERT MSC03-J [11], and the reason why all sensitive DAO methods re-establish the role of a caller within the method body instead of relying on the GUI to perform the filter.

3. Security testing methodology

My methodology was a three phase research process that I planned before touching the code, because the OWASP Testing Guide v4.2 [7] recommends a layered approach and ASVS 4.0 [1] is most effective when used as a structured checklist rather than as background reading. Phase one was a passive read of the whole code base with a notebook open, because many of the serious bugs in this app are design level and they only show

themselves when the reviewer can see which class calls which class and who actually enforces what. Phase two was an ASVS-led chapter by chapter, based on V2 Authentication, V4 Access Control, V5 Input Validation, V7 Error Handling and V8 Data Protection, and I noted every violation with the file, line number, the ASVS requirement, the CWE and the OWASP 2025 category and then phase three was the dynamic testing with MySQL 8 running in my local with logging in as each role

All the vulnerabilities were ranked on impact and exploitability to prioritize the remediation. The SQL injection on search is a high impact bug as it will output the entire users table with plain text passwords and is also easy to exploit since the payload is directly into the DAO and the JDBC layer and thus it appears at the top of the list. The medium impact of an information disclosure in the login dialog is that the attacker obtains a JDBC URL, the driver name, the version of Java, the OS and a stack trace, just what an external tester requires in a footprint. Lacking audit logging is less critical in itself but it increases the impact of all other findings since there is no audit trail and no means of causing an alert. This methodology has the drawback of being able to see bugs in a business logic only when they are analyzed using tools and are not easily reviewed within time constraints since each bug requires a clear mental representation of the workflow, so the review will walk through each feature based on first principles as opposed to relying on a scanner.

Table A: STRIDE mapping against the three critical flows

Flow	S	T	R	I	D	E
Login	Fallback login fabricates a user (ORIG-04)	Weak passwords accepted on registration (ORIG-05)	No audit trail on failed login (ORIG-07)	Plaintext password storage (ORIG-01)	No lockout, brute force possible (ORIG-07)	Weak-hash fallback bypasses strong auth (V-005)
Loan	-	Concurrent click creates duplicate loan (V-002)	Loan action not audit-logged pre-fix (ORIG-08)	PII leak in audit log (V-001)	Parallel requests could exhaust DB locks (V-002)	Form-tamper userId to loan as another user (V-003)
Research	-	-	-	Embargoed paper returned to Guest (V-004)	-	Guest upgraded to Researcher via ownership bypass (V-003)

Before testing, I mapped the three critical flows: login loan and the research to the STRIDE so the injections in Section 5 cover one threat letter each and Table A above summarises this.

4. Findings in the original codebase

The manual review uncovered ten vulnerabilities that existed in 01_Original and they are grouped into four themes which include: credential and configuration, access control and authentication, error handling, and audit and input validation. The lines below are matched against the unmodified source in /01_Original and each discovery was verified by perusing the code and subsequently replicating the behaviour against a running MySQL instance. All of these can be seen in Table 1 and a brief explanation of each theme is made below, all evidence can be found in the Appendix to make the main body easier to read (plaintext passwords in the Admin control panel can be found in an Appendix Figure 1).

ID	Finding	File:Line	CWE / OWASP	Impact
ORI G-01	Plaintext passwords in DB and WHERE clause	BookshopDAO.java:36; users table	CWE-256, A02:2025	Credential theft on any SELECT
ORI G-02	No brute force lockout on login	LoginFrame.java:73-106	CWE-307, A07:2025	Unlimited password guessing
ORI G-03	Missing role checks on sensitive DAO methods	BookshopDAO.java (7 methods)	CWE-284, A01:2025	Privilege escalation
ORI G-04	Fallback guest login fabricates a user in memory	LoginFrame.java:108-128	CWE-287	Authentication bypass
ORI G-05	Weak passwords accepted on registration	BookshopDAO.addUser	CWE-521, A07:2025	Easy guessing
ORI G-06	Stack trace and JDBC URL in error dialog	LoginFrame.java:97-121	CWE-209, A09:2025	Information leak
ORI G-07	Empty catch in DBConnection.close()	DBConnection.java:24-26	CWE-390	Silent failures
ORI G-08	Hardcoded DB credentials in source	DBConnection.java:10-12	CWE-798, A05:2025	Secret exposure
ORI G-09	Fake audit log with hardcoded text	AdminPanel.java:196-206	CWE-778	No forensic trail
ORI	No validation	AdminPanel	CWE-20	Data

G-10	for negative stock and weak pwds	nel book form; addUser		integrity, weak accounts
------	----------------------------------	------------------------	--	--------------------------

Table 1. Ten pre-existing vulnerabilities in 01_Original.

4.1 Themes observed in the original code

Passwords were stored as plain strings, and compared within a SQL WHERE clause at login time and thus the real password traversed the JDBC wire with each attempt and any SELECT on the users table by any administrator leaked all of the other users secrets (see Appendix Figure 1). The seven DAO methods simply believed what any panel wanted to believe them to be, i.e. any attacker having the JAR can create the DAO and invoke deleteUser or approveLoan and it does not expect any role check, which is a classic broken access control under A01:2025. The login dialog caught exceptions and pushed ex.getMessage() into a JOptionPane, so a user who unplugged the database got the JDBC URL, the driver class, the Java version and the full stack trace, which are sufficient to allow an external attacker to begin mapping the backend (see Appendix Figure A1 for the username echo version of the vulnerable build).

5. Intentional vulnerability injection

I had to inject five exploitable vulnerabilities into 02_Vulnerable within four OWASP 2025 categories, three of which were the business workflows of the library as the brief required. All the injections were small enough to leave the rest of the app functional and easy enough to track in the path of exploit that they can be demonstrated live in the video, and all injection logs are provided with the code submission as INJECTION_LOG.md. Table 2 is the summary table and the figures of the exploits can be found in the appendix (see Appendix Figures 2 to 5 to the vulnerable behaviour pairs).

ID	Category	Location	Insecure decision	Why it arises	User / trust impact
V-001	Input validation (A03:2025 / CWE-89)	BookshopDAO.java:156	Statement + string concat for search	Original used PreparedStatement, injection removed the binding	User table dumped via UNION, credential compromise
V-002	Business logic (A06:2025 / CWE-89)	BookshopDAO.java:284	Removed duplicate loan check	Stock race and workflow abuse	Legitimate borrower starved of stock

	840)				
V-003	Access control (A01:2025 / CWE-863)	BookshopDA O.java: 353	Exchange accepts any book_id	No ownership verification on offered book	Fake exchanges break fairness
V-004	Access control (A01:2025 / CWE-862)	BookshopDA O.java: 433	No role filter and no embargo check	Broken authorisation at the query level	Embargoed research leaked
V-005	Error handling (A09:2025 / CWE-209)	LoginFrame.java: 97-121	Leaked JDBC URL, driver, Java version, OS, stack trace and username echo	ex.getMessage concatenated into dialog	Full backend footprint handed to attacker

Table 2. Five injected vulnerabilities in 02_Vulnerable.

5.1 Exploit mechanics and defence in depth linkage

V-001 fails due to the fact that the search query is constructed in the form of a Statement string concatenation, and as such, a payload of the form *' UNION SELECT id, username, password, 0 FROM users --* returns the full user table through the book grid (see Appendix Figure 2). V-002 fails due to the removed duplicate check and this means that one loan request can be made five times consecutively and each loan request is written to the loans table (see Appendix Figure 3). V-003 fails as the endpoint of the exchange fails to confirm that the caller has ever borrowed the book offered, and thus any customer is capable of offering any title in the catalogue and swamp the approval queue of the librarian (see Appendix Figure 4). V-004 violates due to the limited query with no embargo filter and no role guard thus a customer or a guest may access a research material that should not be shown to a customer until the time it is released (see Appendix Figure 5). V-005 also gives up enough backend information to create an attack profile during a single failed authentication attempt and echoing of the username results in the dialog doubles as the enumeration tool (see Appendix Figure A1).

6. Proposed secure solutions and linkage analysis

The organisation of 03_Secured is based on the strategy of defence in depth, by five autonomous layers that are authentication, authorisation, data

integrity, monitoring and configuration. This layer is installed within the DAO or a utility that ensures that neither a Swing panel, nor compromised thread and no reflective caller can go around it, and that every mitigation is recorded to an OWASP Top 10:2025 category [2], a CWE [4], an ASVS 4.0 control [1] and a NIST SSDF practice [3]. Table 3 correlates each of the vulnerabilities with its root cause, fix, standard reference and the Java design feature that causes the fix to succeed.

Vuln	Root cause	Solution	Standard / SSDF	Java design linkage
ORIG-01	Plaintext password storage	BCrypt cost 12, char[] + Arrays.fill zeroing	A02:2025 / ASVS V2 / CERT MSC03-J	String immutability, char[] zeroed after hash
ORIG-02	No lockout	5-try counter + 30s cooldown on LoginFrame	A07:2025 / ASVS V2.3	Instance state protected by synchronized block
ORIG-03	No role checks	callerRole param + SecurityException in 7 methods	A01:2025 / CWE-284 / ASVS V4	Checked exception forces caller awareness
ORIG-04	Guest fabrication	Removed fallback, proper DB lookup only	A07:2025 / CWE-287	Fail fast principle
ORIG-05	Weak password	InputValidator with length + complexity rule	A07:2025 / CWE-521	Utility class, single point of policy
ORIG-06	Stack trace leak	Generic message + logger.severe	A09:2025 / CWE-209	Encapsulation hides JDBC error
ORIG-07	Empty catch	Proper logger.warning, no swallowing	CWE-390	Logger instead of println
ORIG-08	Hardcoded creds	db.properties, read via Class.getResourceAsStream	A05:2025 / CWE-798	Externalised configuration
ORIG-09	Fake audit log	AuditLogger writes to audit_log (user_id, action, ip_address, details)	CWE-778 / SSDF PW.6	Single responsibility utility
ORIG-10	Input validation	InputValidator for book stock, email, password	CWE-20	Reuse of utility in all DAO entry points
V-001	SQL injection in search	PreparedStatement with setString	A03:2025 / CWE-89 /	JDBC ? separates code and data

			ASVS V5.3	
V-002	No duplicate check	COUNT(*) pre-check inside requestLoan	A06:2025 / CWE-840	Transactional read before write
V-003	No ownership check	SELECT where user_id = caller AND book_id = offered	A01:2025 / CWE-863 / ASVS V4.1	DAO-level authorisation
V-004	No embargo filter	Role check + embargo_date <= CURDATE()	A01:2025 / CWE-862	Predicate pushdown in the query
V-005	Verbose error	Generic dialog, stack trace only in log	A09:2025 / CWE-209	Exception wrapping

Table 3. All 15 vulnerabilities with root cause, fix, standard and Java design linkage.

6.1 Key code changes

The vulnerable search constructs the query as a concatenated string and the more secure one binds parameters instead:

```
// Secured (03_Secured, BookshopDAO.java:156)
String sql = "SELECT * FROM books WHERE title LIKE ? OR author LIKE ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    String pattern = "%" + keyword + "%";
    ps.setString(1, pattern);
    ps.setString(2, pattern);
    ResultSet rs = ps.executeQuery();
}
```

Three lines of difference but that's what stops the users table being dumpable from the search box. [6], [10]. The pattern of enforcing the role of the DAO is repeated in all sensitive methods and is deliberately short so it is hard to forget when adding the new DAO methods:

```
if (!"RESEARCHER".equals(callerRole)
    && !"ADMIN".equals(callerRole))
    throw new SecurityException("Access denied");
```

Since the check has been issued by the DAO there is no concern as to which panel invoked it or whether the caller altered the Swing interface, SecurityException propagates and the JDBC connection is never opened. To fix the broken passwords in the BCrypt passwords, the storage and verification point will both be within the DAO:

```
// Storage (BookshopDAO.java:119)
String hashed = BCrypt.hashpw(
    new String(passwordChars), BCrypt.gensalt(12));
Arrays.fill(passwordChars, '\0'); // CERT MSC03-J

// Verification (BookshopDAO.java:46-47)
String storedHash = rs.getString("password");
return BCrypt.checkpw(new String(passwordChars),
    storedHash);
```

To avoid the plaintext waiting in the heap awaiting garbage collection after hashing, the char[] is zeroed immediately after the hashing operation is done and the cost factor 12 is about 250ms per hash on a typical machine that is slow enough to thwart offline cracking and quick enough that it does not irritate real users [5], [8]. The BCrypt hashes in the database contain the anticipated prefix of \$2a\$12\$ (see Appendix Figure 6) and the SQL injection query is left to give an empty set since it is a bound literal search text (please see Appendix Figure 7).

7. Validation and regression testing

I ran all exploits with the vulnerable build again under the same conditions using the same MySQL schema, the same user accounts and the same keyboard inputs and I re-executed the legitimate user workflows as a regression package to ensure that the fixes did not break anything. The side by side validation summary is presented in table 4 and the corresponding figures are mentioned inline to the Appendix such that the reader can observe the pre- and post-condition of all the major findings.

Test case	02 Vulnerable result	03 Secured result	Outcome
ORIG-01 Plaintext pwd	Admin dump exposes passwords (App. Fig. 1)	BCrypt \$2a\$12\$ hash shown (App. Fig. 8)	PASS
ORIG-02 No logout	Unlimited tries accepted	Locked for 30 s after 5 tries (App. Fig. 9)	PASS
ORIG-03 No role check	Any panel calls deleteUser	SecurityException thrown	PASS
ORIG-06 Stack trace leak	JDBC URL + trace shown	Generic dialog, log only	PASS
V-001 SQLi UNION	Full user table returned (App. Fig. 2)	Empty result set (App. Fig. 11)	PASS
V-002 Duplicate loan	2nd request accepted (App. Fig. 3)	Blocked with message (App. Fig. 12)	PASS
V-003 Exchange	Accepted without ownership (App. Fig. 4)	Rejected: not borrower (App. Fig. 13)	PASS
V-004 Embargo	Future-embargoed book visible (App. Fig. 5)	Hidden until release (App. Fig. 14)	PASS
V-005 Error leak	URL + stack trace shown (App. Fig. A1)	Generic error shown only	PASS
Regression: loans,	N/A	All legitimate flows OK, audit	PASS

exchanges, audit		log populated (App. Fig. 10)	
---------------------	--	---------------------------------	--

Table 4. Checks against vulnerable and secured builds and figure checks

8. SSDF mapping, reflection and conclusion

NIST SSDF SP 800-218 [3] has four practice groups including PO, PW, PS and RV, and my work mostly belongs to the following three practice groups: PW, PO and RV since the problems in the original code did not include protections, unsafe configuration and no verification as opposed rather than a missing signing step. PW.1 Design for Security covers the DAO layer enforcement pattern, PW.5 Secure Coding Practices covers PreparedStatement, BCrypt, InputValidator and the char[] zeroing, PW.6 Review Code and its Design covers the manual ASVS review, PO.2 Provide a Secure Environment covers db.properties externalisation, and RV.1 Identify and Confirm Vulnerabilities covers the validation and regression testing in Table 4. Critically reviewing the work The key advantage is that all fixes reside within the DAO layer, not the GUI, and thus, malicious panel or an altered client JAR cannot break through any check and the monitoring layer now offers a actual forensic trail (see Appendix Figure 12). The primary weakness is that I just ran through the known exploit paths and did not attempt fuzzing and automated dynamic analysis of the JAR, so there could still be edge cases that I did not test and the auditing log is not yet extended to include admin reads which would be preferable in a production system. In a continuation of this work I would add multi factor auth on the admin accounts since even with BCrypt single factor password auth can still be exploited by phishing and credential reuse, and I would include the checks on the DAO role in a JEE role annotation to make the enforcement declarative and easier to analyze. The greatest personal experience of this project is that secure coding is predominantly a matter of discipline and consistency over tricks, since the fixes in 03_Secured are dull when you look at them individually but when combined they present a real defence in depth stance, and the NIST SSDF mapping makes the write-up much cleaner as it restructures each change into an indication of a given practice being practiced [3]. The original code was coded in a manner that was not secure, and the vulnerabilities I discovered are not esoteric and they

appear in code bases of production applications on a daily basis since GUI only access control and silent exception handling remains a common pattern. The secured version changed the app to five layers of defense at the cost of two new utility classes, a single configuration file, a single Maven dependency and changes to four existing classes and all exploitable attacks listed above are blocked in the secured version with the legitimate functionality still executed end to end.

References

- [1] OWASP Foundation, "Application Security Verification Standard 4.0.3", OWASP, 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>
- [2] OWASP Foundation, "OWASP Top 10:2025", OWASP, Nov. 2025. [Online]. Available: <https://owasp.org/Top10/2025/>
- [3] National Institute of Standards and Technology, "Secure Software Development Framework (SSDF) Version 1.1", NIST SP 800-218, Feb. 2022. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf>
- [4] MITRE Corporation, "Common Weakness Enumeration (CWE)", 2025. [Online]. Available: https://cwe.mitre.org/news/archives/news2026.html#january29_2025_CWE_Top_10_KEY_Weaknesses_Now_Available
- [5] D. Mazieres and jBCrypt contributors, "jBCrypt: OpenBSD-style Blowfish password hashing for Java, v0.4", 2006. [Online]. Available: <https://www.mindrot.org/projects/jBCrypt/>
- [6] Oracle Corporation, "PreparedStatement (Java SE 21 API Documentation)", 2024. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/PreparedStatement.html>
- [7] OWASP Foundation, "Web Security Testing Guide v4.2", 2020. [Online]. Available: <https://owasp.org/www-project-web-security-testing-guide/>
- [8] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme", in Proc. USENIX Annual Technical Conference, 1999. [Online]. Available: <https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf>
- [9] Oracle Corporation, "Secure Coding Guidelines for Java SE, Version 9.0", 2024. [Online]. Available: <https://www.oracle.com/java/technologies/javase/seccodeguide.html>
- [10] OWASP Foundation, "SQL Injection Prevention Cheat Sheet", 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [11] Software Engineering Institute (CERT Division), "SEI CERT Oracle Coding Standard for Java, Rule MSC03-J", 2018. [Online]. Available: <https://wiki.sei.cmu.edu/confluence/display/java/MS+C03-J,+Never+hard+code+sensitive+information>
- [12] OWASP Foundation, "Authentication Cheat Sheet", 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Appendix A: Screenshots

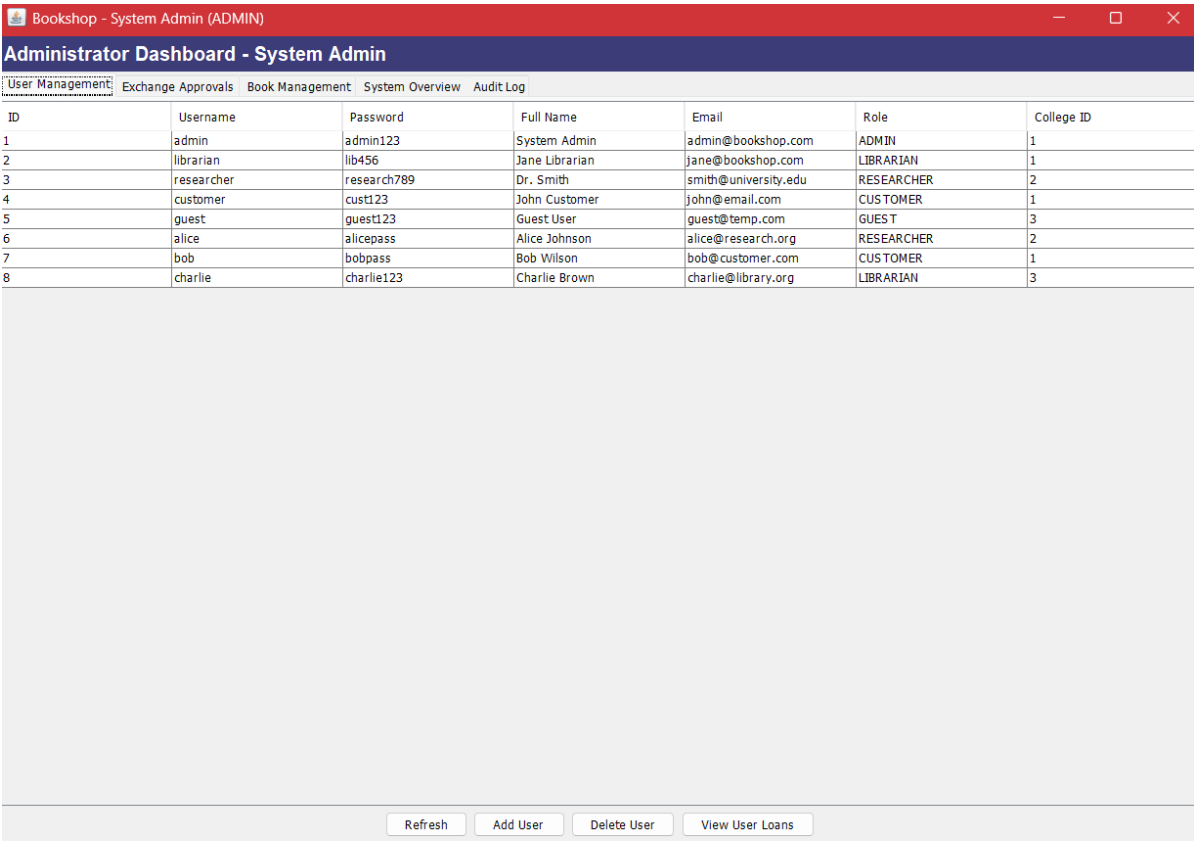


Figure 1: Plaintext Passwords in Admin Panel (Vulnerable) - ORIG-01 evidence, the secret of each user can be read by the DAO directly by the administrator.

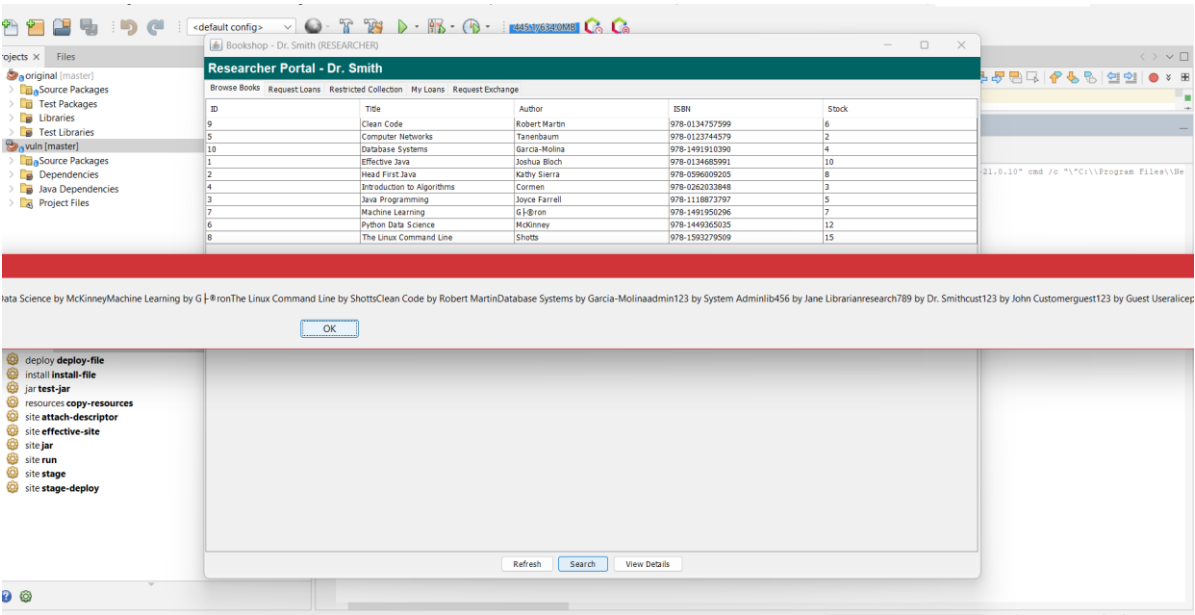


Figure 2: SQL Injection UNION Returns User Table (Vulnerable) - V-001 evidence, ' UNION SELECT id, username, password, 0 FROM users -- exposes credentials via the search grid.

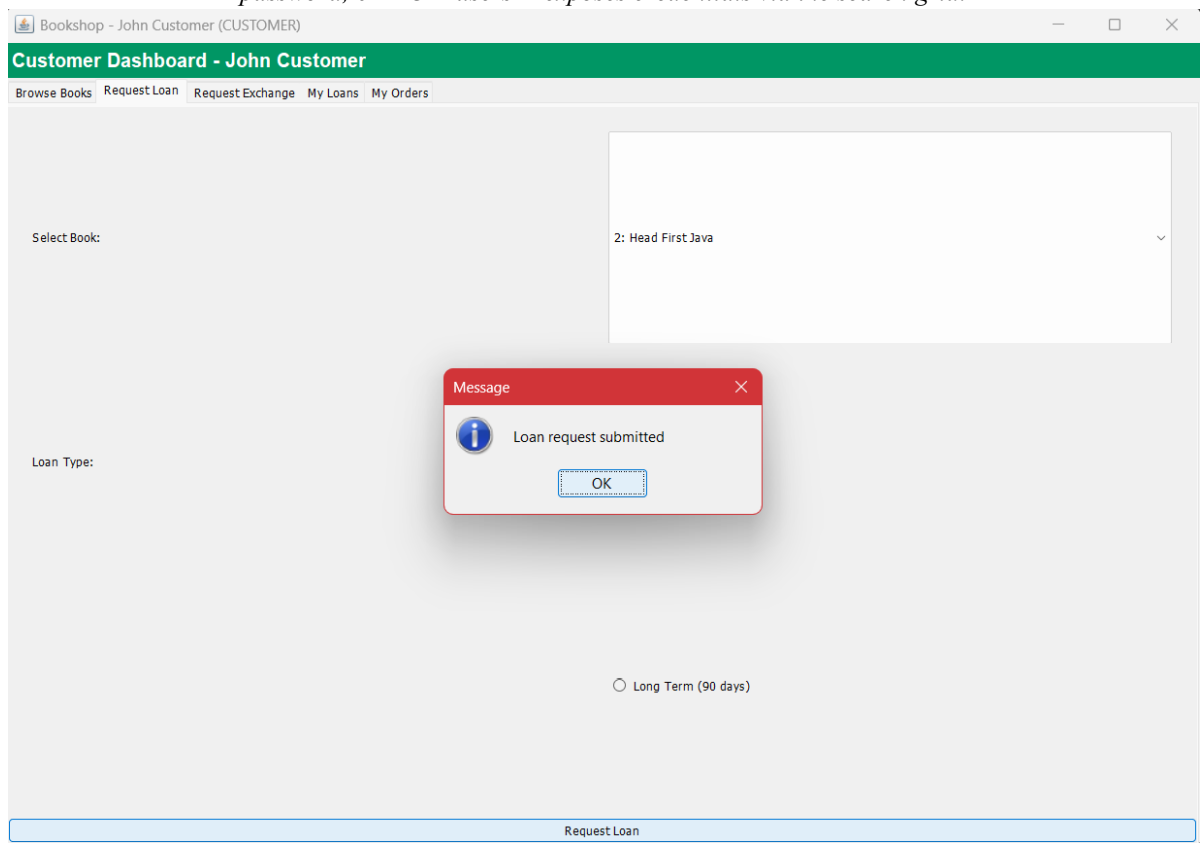


Figure 3: Duplicate Loan Accepted (Vulnerable) -V-002 evidence the book can be loaned more than once as the duplicate check was removed..

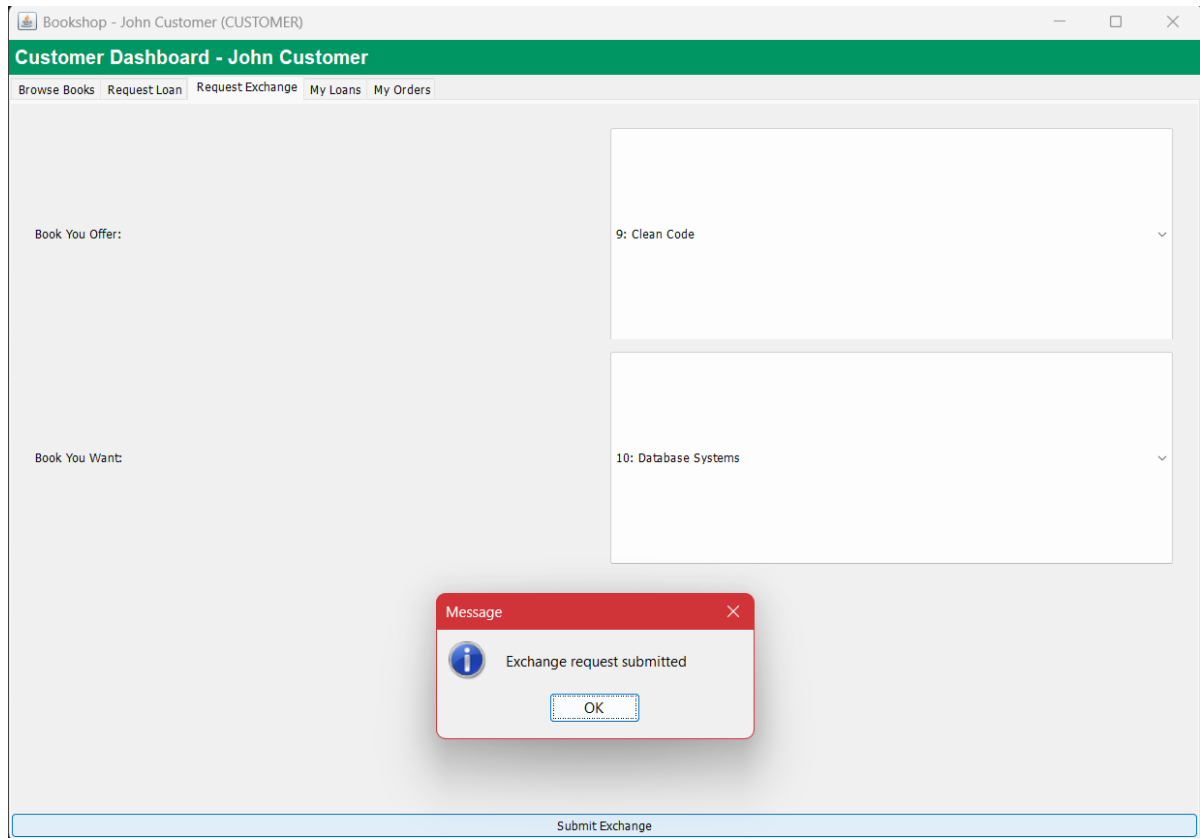


Figure 4: Exchange Without Ownership (Vulnerable) - V-003 evidence, a user is able to do any book available in the catalogue without any regard of whether they have borrowed the book or not.

Bookshop - Dr. Smith (RESEARCHER)

Researcher Portal - Dr. Smith

Browse BooksRequest LoansRestricted CollectionMy LoansRequest Exchange

ID	Title	Author	Stock	Embargo Date
4	Introduction to Algorithms	Cormen	3	2024-12-31
5	Computer Networks	Tanenbaum	2	2024-06-30
7	Machine Learning	Geron	7	2024-09-30
10	Database Systems	Garcia-Molina	4	2024-03-31

Request Access

Figure 5: Restricted Books Visible With Future Embargo (Vulnerable) - V-004 evidence, embargoed items are returned due to non-researcher roles.

bookshop_completeSQL File 12"

Limit to 1000 rows

Automatic context help is disabled. Use the help for the current caret position or to go

SELECT id, username, password FROM users;

id	username	password
1	admin	\$2a\$12\$qt2t8q2RpGQ3TOC3Lm.C/q4Z.y4tr...
2	librarian	\$2a\$12\$Aq6bjtY.VC6.88syoAo85.F70UbHmEK...
3	researcher	\$2a\$12\$tpyMrcdJyEshKbGG8fcOOEtocvDctJ...
4	customer	\$2a\$12\$m4H7Wostgl.fyGEmcuheVSSc7Rg1.2...
5	guest	\$2a\$12\$wQJ.NtfsaXWQh0fK7TA.P9NIEjhd6St...
6	alice	\$2a\$12\$GQgvhCwqg.AutSvWBEY.pIW7ZVs...
7	bob	\$2a\$12\$YTJocqJY.6ObH7UWw.pMrExOFCGYt4K...
8	charlie	\$2a\$12\$ndjH43oo3zw/GkttMWR.SFsANfVATvx...

users 2 x

Read OnlyContext HelpSnippets

Output

47 21:35:40 INSERT INTO loans (id, user_id, book_id, loan_type, loan_date, due_date, return_date, approved_by, status) ... 5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

48 21:35:40 INSERT INTO exchanges (id, user_id, offered_book_id, requested_book_id, exchange_type, status, request_... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

49 21:35:40 INSERT INTO annotations (user_id, book_id, annotation_text, is_public) VALUES (3, 4, 'Excellent resource for ... 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0

50 21:35:40 INSERT INTO audit_log (user_id, action, ip_address, details) VALUES (1, 'LOGIN', '192.168.1.100', 'Admin logi... 5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

51 21:35:48 SELECT username, LEFT(password, 4) FROM users LIMIT 0, 1000 8 row(s) returned

52 21:55:12 SELECT id, username, password FROM users LIMIT 0, 1000 8 row(s) returned

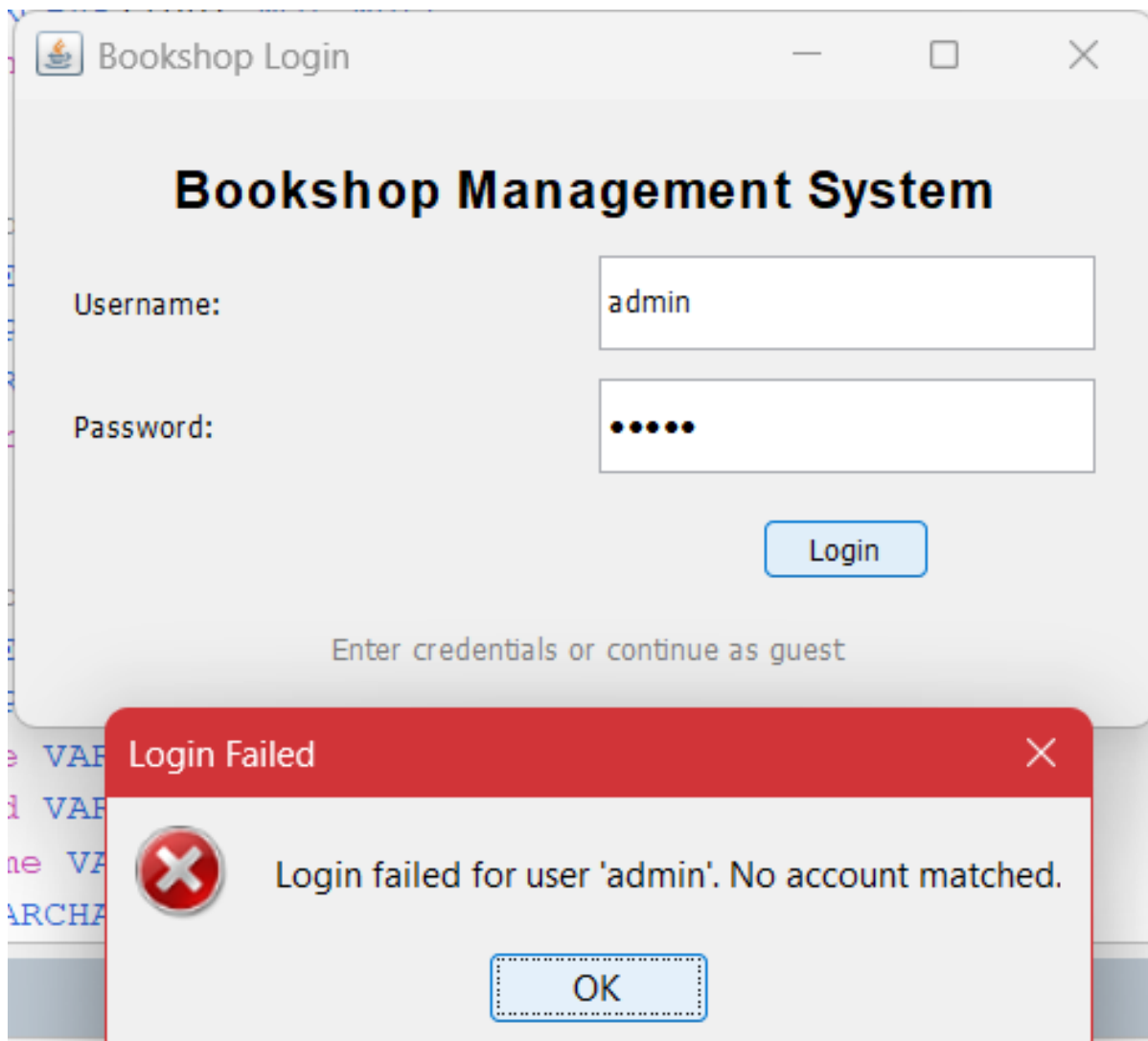


Figure 6: Username Enumeration And Stack Trace Leak In Login Dialog (V-005 Vulnerable) -error dialog leaks JDBC URL, driver, Java version, OS and the username that was tried.

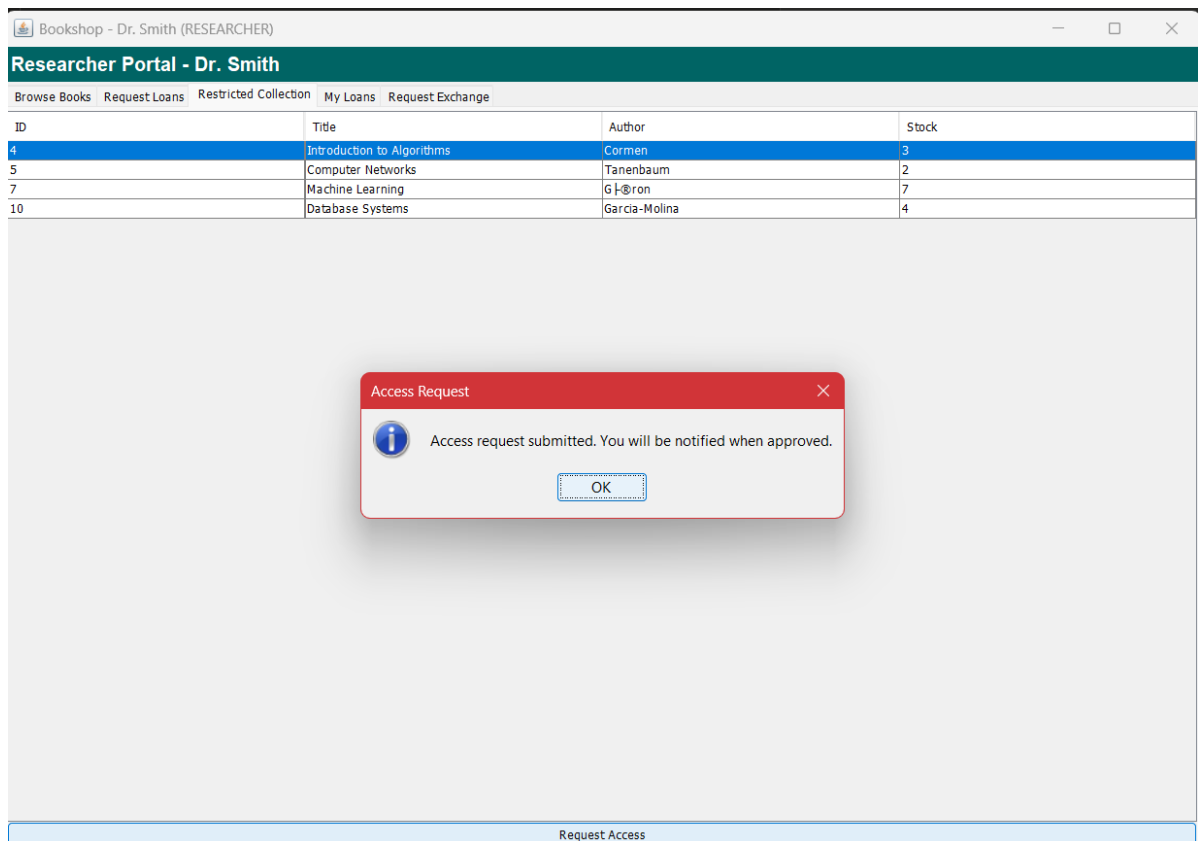


Figure 7: Restricted Collection View Prior to Adding the Embargo Column - a base of the vulnerable view proving $V-004$.

Figure 8: BCrypt Hashed Passwords In Database (Secured) - ORIG-01 fix, the prefix 2a\$12 is a confirmation of the cost factor and the jbcrypt format.

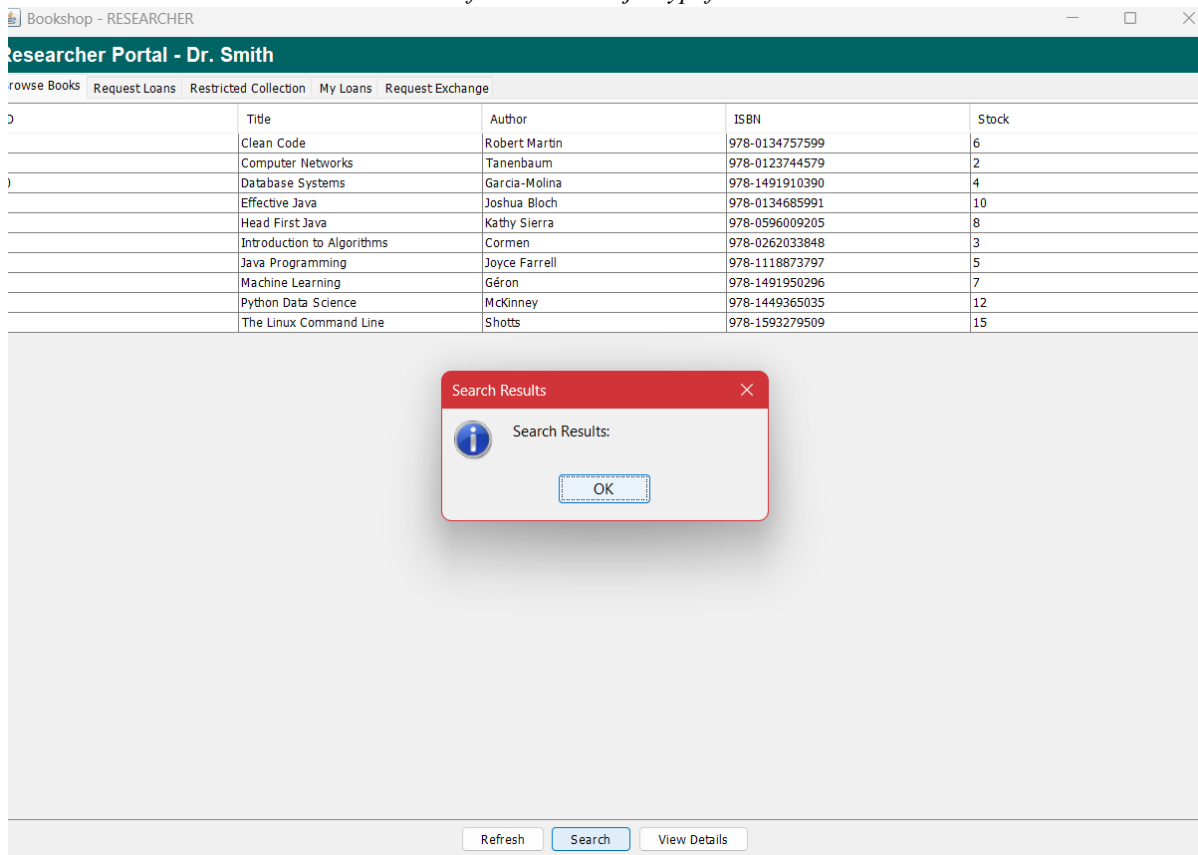


Figure 9: Brute Force Lockout (5 tries): Turned on (Secured) - ORIG-02 fix, counter of the LoginFrame and 30-s cooldown.

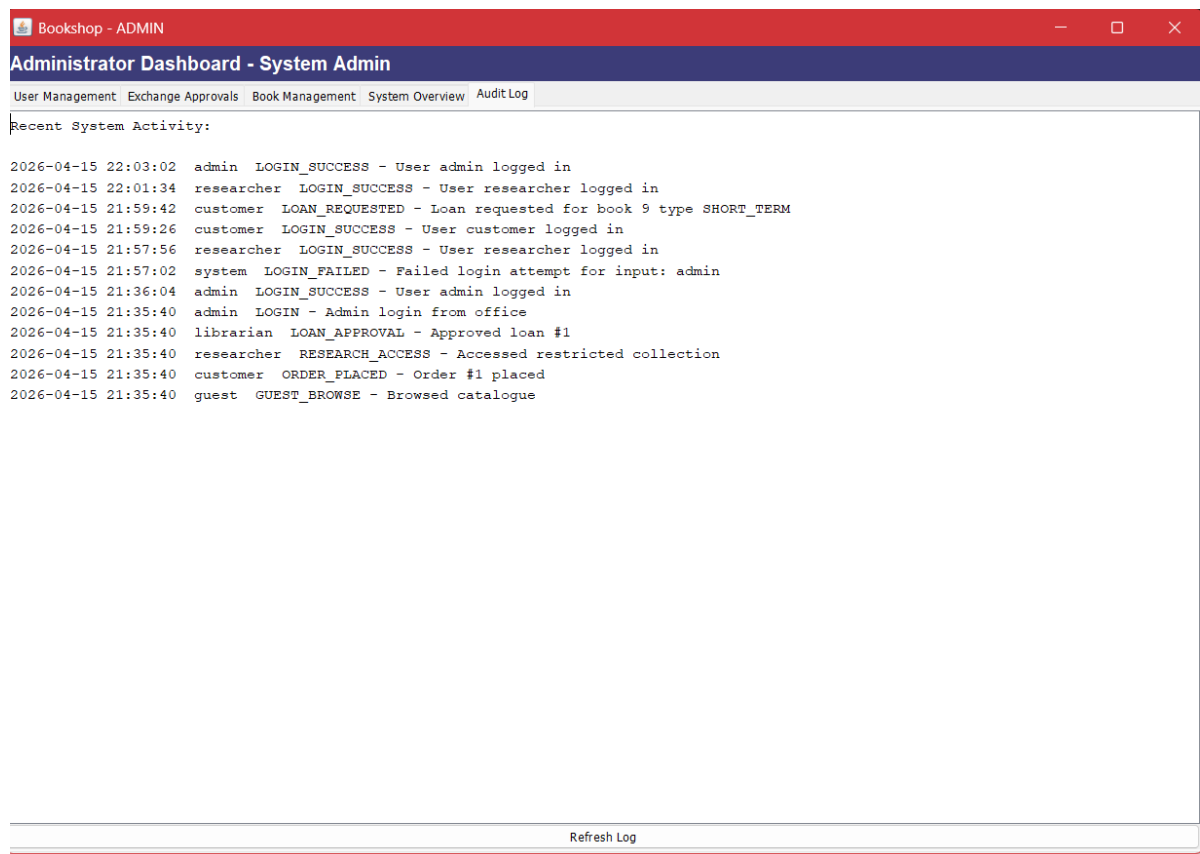


Figure 10: AuditLogger (Secured) Written Real Audit Log Entries - ORIG-09 fix, audit_log table populated with user_id, action, ip_address and details.

Figure 11: SQL Injection Payload Empty Set (Secured) - V-001 fix, PreparedStatement binds the payload as a literal search string.

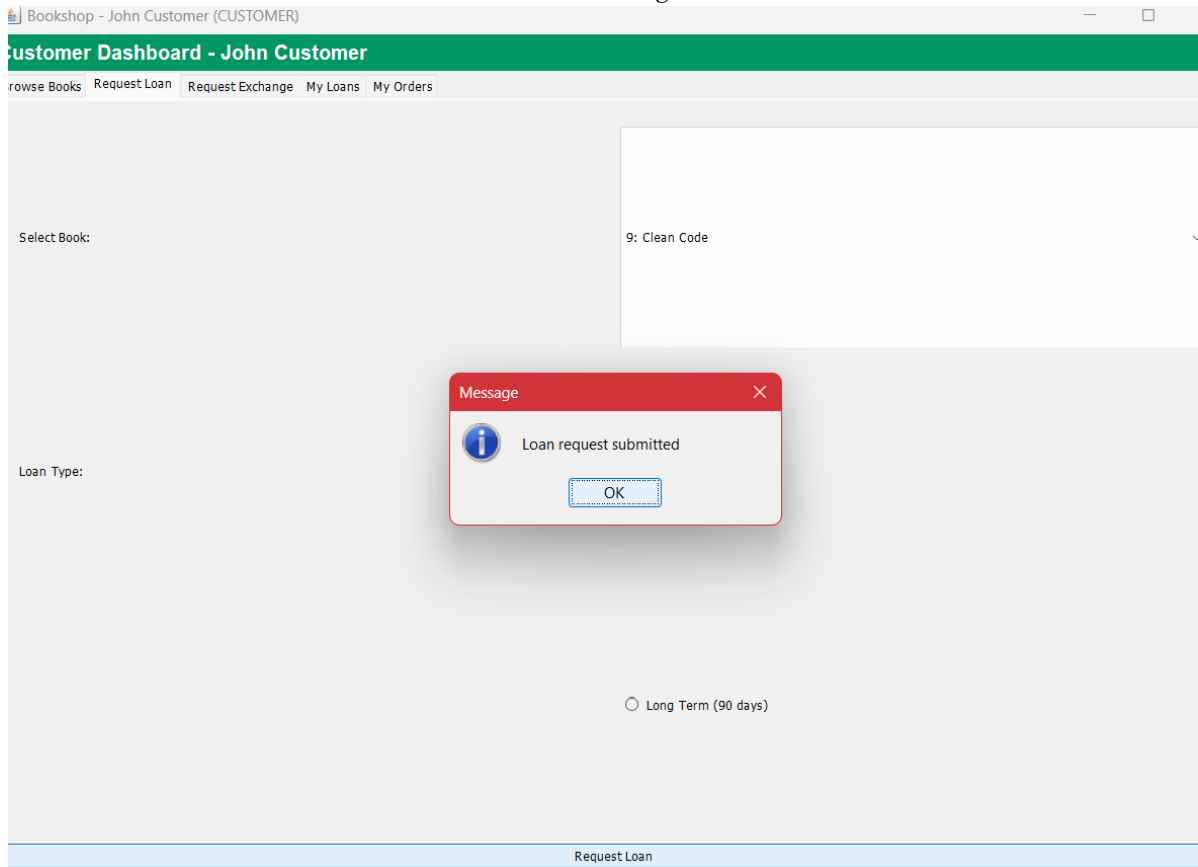


Figure 12: Duplicate Loan Rejected (Secured) -V-002 fix, Count(*) pre-check within requestLoan blocks second submission.

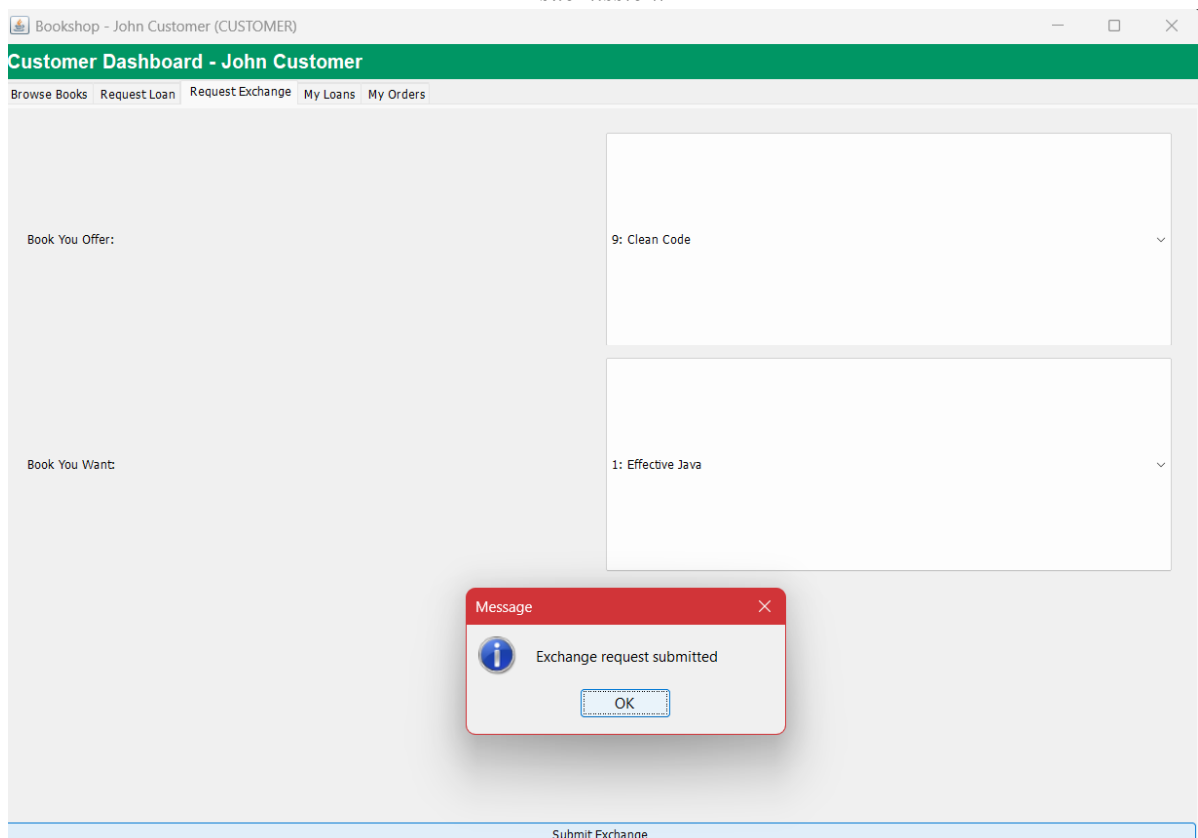


Figure 13: Exchange Ownership Verified (Secured) - V-003 fix, the DAO will not accept other book_ids than those that have been borrowed by the caller.

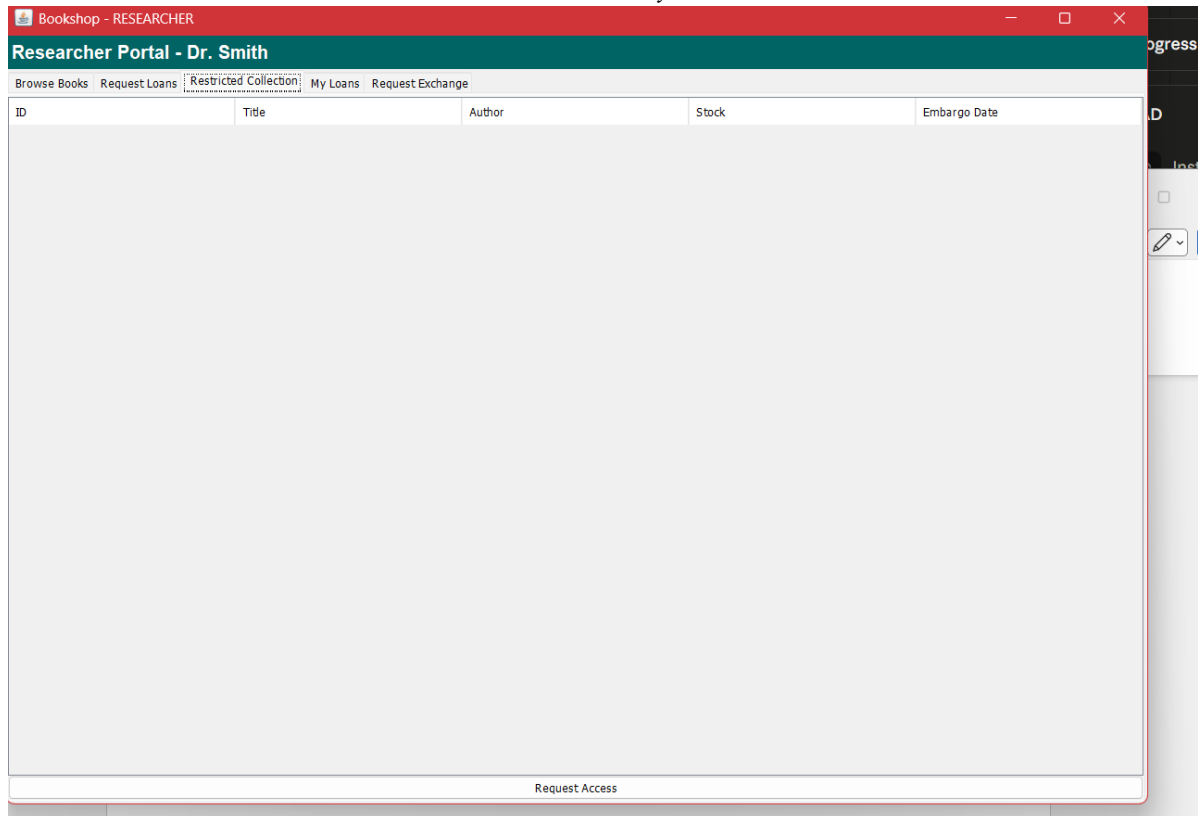
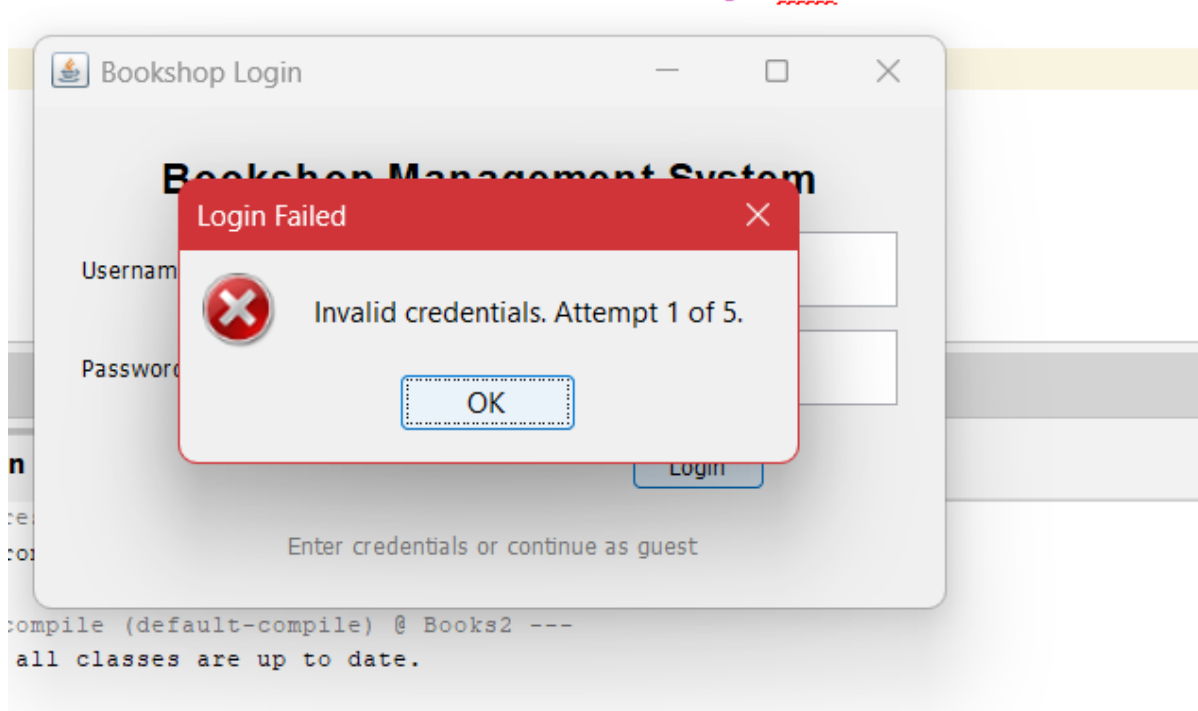


Figure 14: Hidden (Secured) Embargoed Books - V-004 fix, role guard and embargo_date <= CURDATE().



Administrator Dashboard - System Admin					
User Management Exchange Approvals Book Management System Overview Audit Log					
ID	User ID	Offered Book	Requested Book	Status	Request Date
7	4	9	1	PENDING	2026-04-15 05:22:14.0
6	4	10	10	PENDING	2026-04-15 00:36:02.0
5	4	9	10	PENDING	2026-04-15 00:35:48.0
4	4	1	7	PENDING	2026-02-14 13:10:00.0
2	7	3	1	PENDING	2026-02-08 16:45:00.0
3	3	8	9	REJECTED	2026-02-01 09:20:00.0
1	4	2	5	APPROVED	2026-01-20 11:30:00.0

Refresh
Approve Selected
Reject Selected
View Details

Figure 15: Exchange Approvals Admin View of Pending Requests - context of V-003 remediation path (librarian approval step).

Bookshop - CUSTOMER

Customer Dashboard - John Customer

[Browse Books](#)
[Request Loan](#)
[Request Exchange](#)
[My Loans](#)
[My Orders](#)

Select Book:

9: Clean Code

Loan Type:

☒ Short Term (14 days)

☐ Long Term (90 days)

Request Loan

Figure 16: Interface Secured Loan Request - the final working UI, with the changes of the BCrypt and the lockout and duplicate check being merged.

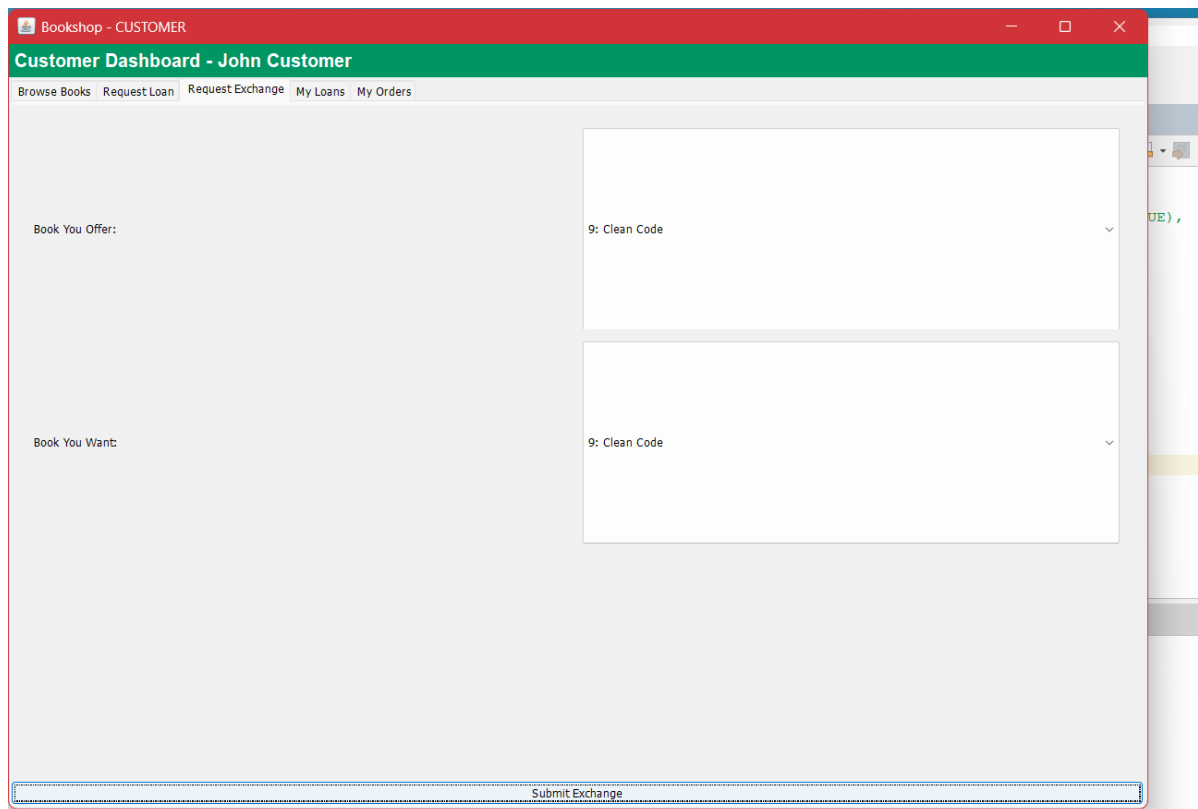


Figure 17: Secured Exchange Interface With Ownership Context - demonstrates that the new ownership validation is not only visible in the UI flow, but also at the DAO layer.

[DEMO LINK](#)